TCC 2026

Relaxed Modular PCS from Arbitrary PCS

and Applications to SNARKs for Integers

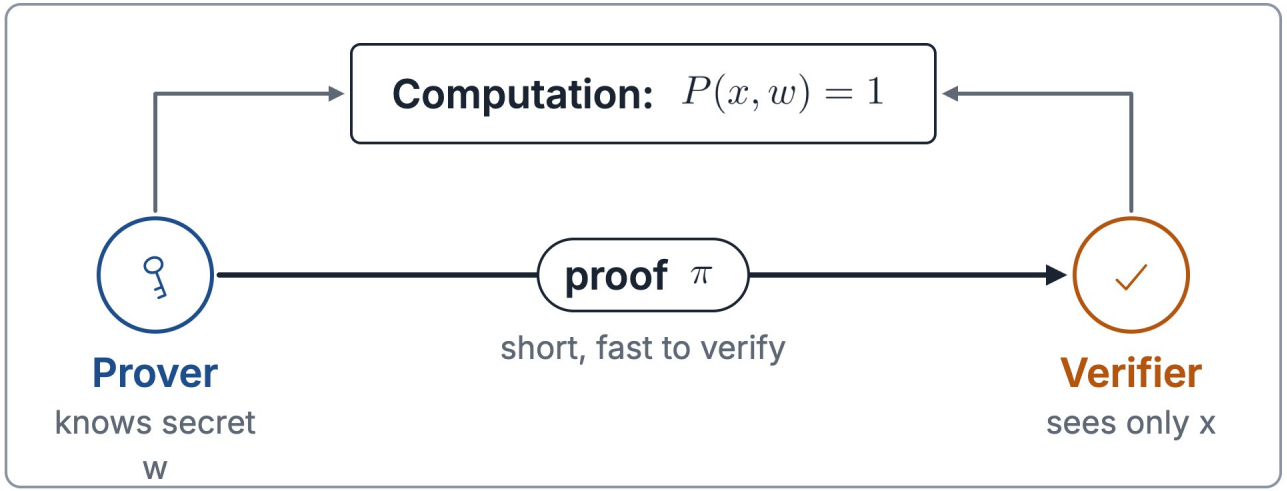
Alireza Shirzad* · Sriram Sridhar* · Dimitrios Papadopoulos · Charalampos Papamanthou

University of Pennsylvania · UC Berkeley · HKUST · Lagrange Labs and Yale University

* Work partially done while at Lagrange Labs

These slides were created by AI under human supervision.

SNARKs and the computations they prove



Machine learning floating and fixed point

CPU operations the ring $\mathbb{Z}/2^k\mathbb{Z}$ k-bit words

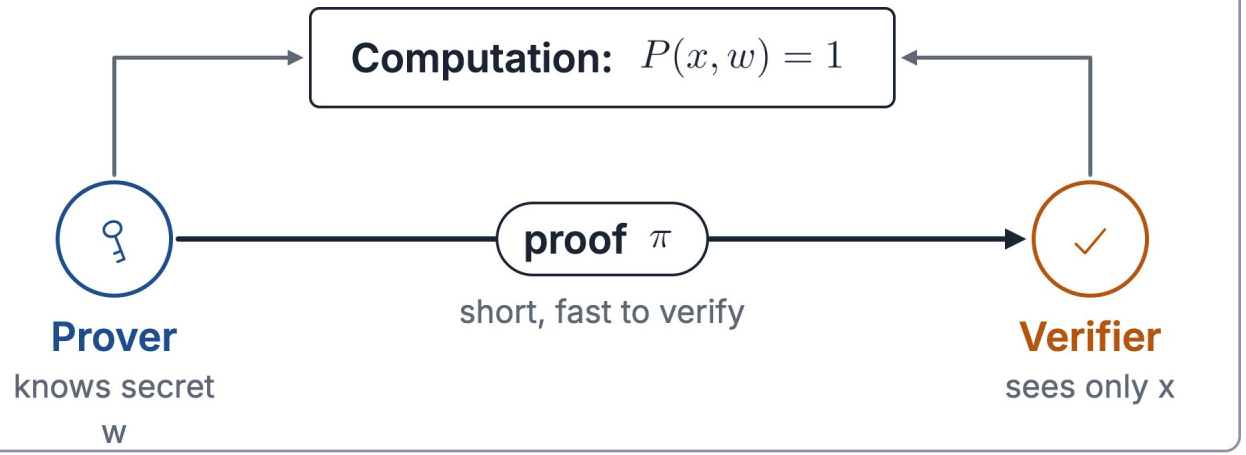
The gap

Finite-field languages
R1CS, circuit satisfiability, Plonkish

What we want to prove

How P is expressed

SNARKs and the computations they prove



How P is expressed

The computation P is usually expressed in an NP-complete language over a finite field.

Examples: R1CS, circuit satisfiability, Plonkish.

Machine learning floating and fixed point

CPU operations the ring $\mathbb{Z}/2^k\mathbb{Z}$ k-bit words

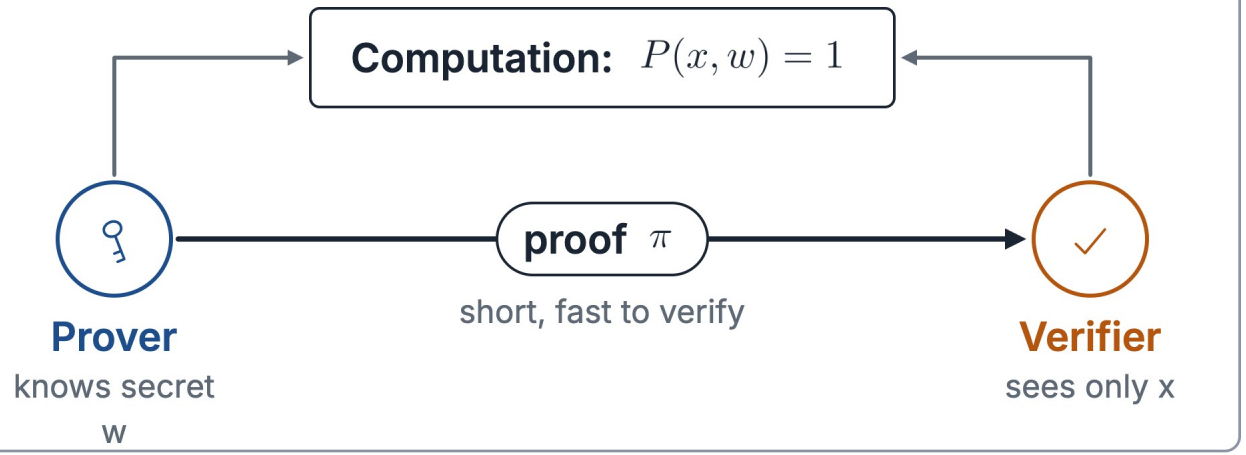
The gap

Finite-field languages
R1CS, circuit satisfiability, Plonkish

What we want to prove

How P is expressed

SNARKs and the computations they prove



How P is expressed

The computation P is usually expressed in an NP-complete language over a finite field.

Examples: R1CS, circuit satisfiability, Plonkish.

Machine learning floating and fixed point

CPU operations the ring $\mathbb{Z}/2^k\mathbb{Z}$ k-bit words

The gap

Finite-field languages
R1CS, circuit satisfiability, Plonkish

What we want to prove

How P is expressed

The price of emulation inside a field

A huge blowup

$2^5\times$

for basic 32-bit CPU arithmetic

$>2^{12}\times$

for cryptographic computations such as RSA

RSA and bug figures as reported in Limber, CXNB26

The price of emulation inside a field

A huge blowup

$2^5\times$

for basic 32-bit CPU arithmetic

$>\ 2^{12}\times$

for cryptographic computations such as RSA

Soundness pitfalls

Every range check and carry relation is written by hand. One omission breaks soundness.

9 of 27 critical bugs in real-world ZK systems came from non-native arithmetic

$450k bug bounty paid for one non-native arithmetic soundness bug

RSA and bug figures as reported in Limber, CXNB26

The price of emulation inside a field

A huge blowup

$2^5\times$

for basic 32-bit CPU arithmetic

$> 2^{12}\times$

for cryptographic computations such as RSA

Soundness pitfalls

Every range check and carry relation is written by hand. One omission breaks soundness.

9 of 27 critical bugs in real-world ZK systems came from non-native arithmetic

$450k bug bounty paid for one non-native arithmetic soundness bug

Integer constraints avoid the blowup

RSA: one constraint per multiplication, $a \cdot b \equiv c \pmod{N}$

For two RSA accumulator updates:

12,796
integer constraints

10,700,000
field constraints

RSA and bug figures as reported in Limber, CXNB26

The price of emulation inside a field

A huge blowup

$2^5\times$

for basic 32-bit CPU arithmetic

$> 2^{12}\times$

for cryptographic computations such as RSA

Soundness pitfalls

Every range check and carry relation is written by hand. One omission breaks soundness.

9 of 27 critical bugs in real-world ZK systems came from non-native arithmetic

$450k bug bounty paid for one non-native arithmetic soundness bug

Integer constraints avoid the blowup

RSA: one constraint per multiplication, $a \cdot b \equiv c \pmod{N}$

For two RSA accumulator updates:

12,796 integer constraints

10,700,000 field constraints

Our goal: design SNARKs that work directly over the integers.

RSA and bug figures as reported in Limber, CXNB26

The blueprint for SNARKs over the integers

First, the difficulties

No inverses

Integers have no multiplicative inverses, which complicates extraction

Hard to commit

Commitment schemes for integers are scarce

Hard to prove

Proving statements over integers efficiently is difficult

The blueprint for SNARKs over the integers

First, the difficulties

No inverses
Integers have no multiplicative inverses, which complicates extraction

Hard to commit
Commitment schemes for integers are scarce

Hard to prove
Proving statements over integers efficiently is difficult

The CHA24 framework

Modular PIOP (Mod-PIOP)
Polynomial interactive oracle proof: the information-theoretic part

+

Modular PCS (Mod-PC)
Polynomial commitment scheme: the cryptographic part

→

SNARK over the integers
The result

The blueprint for SNARKs over the integers

First, the difficulties

No inverses
Integers have no multiplicative inverses, which complicates extraction

Hard to commit
Commitment schemes for integers are scarce

Hard to prove
Proving statements over integers efficiently is difficult

The CHA24 framework

Modular PIOP (Mod-PIOP)
Polynomial interactive oracle proof: the information-theoretic part

+

Modular PCS (Mod-PC)
Polynomial commitment scheme: the cryptographic part

→

SNARK over the integers
The result

😊 Good news: the Mod-PIOP can be copied
The information-theoretic part comes from the finite-field world. Sumcheck-based PIOPs, for example, work here essentially as they are.

The blueprint for SNARKs over the integers

First, the difficulties

No inverses
Integers have no multiplicative inverses, which complicates extraction

Hard to commit
Commitment schemes for integers are scarce

Hard to prove
Proving statements over integers efficiently is difficult

The CHA24 framework

Modular PIOP (Mod-PIOP)
Polynomial interactive oracle proof: the information-theoretic part

+

Modular PCS (Mod-PC)
Polynomial commitment scheme: the cryptographic part

→

SNARK over the integers
The result

😊 Good news: the Mod-PIOP can be copied
The information-theoretic part comes from the finite-field world. Sumcheck-based PIOPs, for example, work here essentially as they are.

😞 Bad news: the Mod-PC cannot
The cryptographic part dictates the security and the assumptions of the scheme, and a normal PCS cannot simply be reused for it.

The Mod-PC and its landscape before this work

$\mathsf{Setup}(\lambda, N, B) \rightarrow (\mathsf{ck}, \mathsf{vk})$

$\mathsf{Commit}(\mathsf{ck}, f) \rightarrow \mathsf{cm}$

$\mathsf{Open}(\mathsf{ck}, f, y, q) \rightarrow (z, \pi)$ *q*: a random prime sampled after Commit.

The Mod-PC and its landscape before this work

$\mathsf{Setup}(\lambda, N, B) \rightarrow (\mathsf{ck}, \mathsf{vk})$

$\mathsf{Commit}(\mathsf{ck}, f) \rightarrow \mathsf{cm}$

$\mathsf{Open}(\mathsf{ck}, f, y, q) \rightarrow (z, \pi)$ *q*: a random prime sampled after Commit.

The claim π proves: $f(y) \equiv z \pmod{q}$

The Mod-PC and its landscape before this work

$\mathsf{Setup}(\lambda, N, B) \rightarrow (\mathsf{ck}, \mathsf{vk})$

$\mathsf{Commit}(\mathsf{ck}, f) \rightarrow \mathsf{cm}$

$\mathsf{Open}(\mathsf{ck}, f, y, q) \rightarrow (z, \pi)$ *q*: a random prime sampled after Commit.

≠

Field-agnostic PCS

Any field works, but it is fixed before Commit.

The claim π proves: $f(y) \equiv z \pmod{q}$

The Mod-PC and its landscape before this work

$\mathsf{Setup}(\lambda, N, B) \rightarrow (\mathsf{ck}, \mathsf{vk})$

$\mathsf{Commit}(\mathsf{ck}, f) \rightarrow \mathsf{cm}$

$\mathsf{Open}(\mathsf{ck}, f, y, q) \rightarrow (z, \pi)$ *q*: a random prime sampled after Commit.

$\neq$

Field-agnostic PCS

Any field works, but it is fixed before Commit.

The claim π proves: $f(y) \equiv z \pmod{q}$

The PCS literature: KZG10, PST13, ZGKPP17, XZZPS19, MBKM19, CHMMVW20, BDFG20, BDFG21, BMMTV21, BCHO22, KT23, CGGJMT23, EG25, GPS25, BCCGP16, BBBPWM18, WTSTW18, BGH19, BCMS20, SL20, Lee21, BFS20, BHRRS21, AGLMS23, SB23, BMSS25, AHIV17, BBHR18, KPV19, BGKS20, BCIKS20, ZXZS20, BFHVXZ20, BCG20, XZS22, GLSTW23, CBBZ23, DP23, DP24, ZCF24, ACFY24, ACFY25, HLP24, BSU24, ZGD+24, GLHQTZ25, BCFRRZ25, BMMS25, NA25, BFRW25, ACLMT22, WW23, BS23, BCS23, FMN24, AFLN24, CMNW24, HSS24, NS24, NOZ26, ...

The Mod-PC and its landscape before this work

$\mathsf{Setup}(\lambda, N, B) \rightarrow (\mathsf{ck}, \mathsf{vk})$

$\mathsf{Commit}(\mathsf{ck}, f) \rightarrow \mathsf{cm}$

$\mathsf{Open}(\mathsf{ck}, f, y, q) \rightarrow (z, \pi)$ *q*: a random prime sampled after Commit.

≠

Field-agnostic PCS

Any field works, but it is fixed before Commit.

The claim π proves: $f(y) \equiv z \pmod{q}$

The PCS literature: KZG10, PST13, ZGKPP17, XZZPS19, MBKM19, CHMMVW20, BDFG20, BDFG21, BMMTV21, BCHO22, KT23, CGGJMT23, EG25, GPS25, BCCGP16, BBBPWM18, WTSTW18, BGH19, BCMS20, SL20, Lee21, BFS20, BHRRS21, AGLMS23, SB23, BMSS25, AHIV17, BBHR18, KPV19, BGKS20, BCIKS20, ZXZS20, BFHVXZ20, BCG20, XZS22, GLSTW23, CBBZ23, DP23, DP24, ZCF24, ACFY24, ACFY25, HLP24, BSU24, ZGD+24, GLHQTZ25, BCFRRZ25, BMMS25, NA25, BFRW25, ACLMT22, WW23, BS23, BCS23, FMN24, AFLN24, CMNW24, HSS24, NS24, NOZ26, ...

The Mod-PC literature: surprisingly, only two constructions

Scheme	Built on	PQ	Unbounded integers	Extracts	Prover	Verifier	Proof
CHA24	Unknown-order groups	✗	✓	Integers	BN log N	log N · log² B	log² B
Zip, GWV+25	Hash-based codes	✓	✗	Rationals	N · B	√N · B	√N · B

RESEARCH QUESTION

Can we keep all of that PCS literature instead of reinventing the wheel? **Is there a black-box way to build a Mod-PC from any field PCS?**

RESEARCH QUESTION

> Can we keep all of that PCS literature instead of reinventing the wheel? **Is there a black-box way to build a Mod-PC from any field PCS?**

MAIN RESULT

We introduce Qāren, a black-box compiler from any given PCS to a Mod-PC.

RESEARCH QUESTION

> Can we keep all of that PCS literature instead of reinventing the wheel? **Is there a black-box way to build a Mod-PC from any field PCS?**

MAIN RESULT

We introduce Qāren, a black-box compiler from any given PCS to a Mod-PC.

Scheme	Built on	PQ	Unbounded integers	Extracts	Prover	Verifier	Proof
CHA24	Unknown-order groups	✗	✓	Integers	BN log N	log N · log² B	log² B
Zip, GWV+25	Hash-based codes	✓	✗	Rationals	N · B	√N · B	√N · B
Qāren (this work) + TensorSwitch, BFRW25	Hash-based tensor codes	✓	✗	Rationals	BN log(BN)	log(B + N)	log(B + N)

The insight: evaluation as an inner product

We have a polynomial (multilinear or univariate), represented as a vector of coefficients or evaluations:

$$\mathbf{f} = (f_0, f_1, \ldots, f_{N-1})$$

The insight: evaluation as an inner product

We have a polynomial (multilinear or univariate), represented as a vector of coefficients or evaluations:

$$\mathbf{f} = (f_0, f_1, \ldots, f_{N-1})$$

Evaluating it at a point z is an inner product with a structured vector $\mathbf{g}$:

$$f(z) \;=\; \langle \mathbf{f}, \mathbf{g} \rangle$$

The insight: evaluation as an inner product

We have a polynomial (multilinear or univariate), represented as a vector of coefficients or evaluations:

$$\mathbf{f} = (f_0, f_1, \ldots, f_{N-1})$$

Evaluating it at a point z is an inner product with a structured vector $\mathbf{g}$:

$$f(z) \;=\; \langle \mathbf{f}, \mathbf{g} \rangle$$

$$\mathbf{g} = (1, z, z^2, \ldots, z^{N-1}) \;\; \text{(univariate)}, \qquad \mathbf{g} = \bigotimes_{j=1}^{\log N} (1, z_j) \;\; \text{(multilinear)}$$

The insight: evaluation as an inner product

We have a polynomial (multilinear or univariate), represented as a vector of coefficients or evaluations:

$$\mathbf{f} = (f_0, f_1, \ldots, f_{N-1})$$

Evaluating it at a point z is an inner product with a structured vector $\mathbf{g}$:

$$f(z) \;=\; \langle \mathbf{f}, \mathbf{g} \rangle$$

$$\mathbf{g} = (1, z, z^2, \ldots, z^{N-1}) \;\; \text{(univariate)}, \qquad \mathbf{g} = \bigotimes_{j=1}^{\log N} (1, z_j) \;\; \text{(multilinear)}$$

So it suffices to prove an inner product over the integers.

The insight: evaluation as an inner product

We have a polynomial (multilinear or univariate), represented as a vector of coefficients or evaluations:

$$\mathbf{f} = (f_0, f_1, \ldots, f_{N-1})$$

Evaluating it at a point z is an inner product with a structured vector $\mathbf{g}$:

$$f(z) \;=\; \langle \mathbf{f}, \mathbf{g} \rangle$$

$$\mathbf{g} = (1, z, z^2, \ldots, z^{N-1}) \;\; \text{(univariate)}, \qquad \mathbf{g} = \bigotimes_{j=1}^{\log N} (1, z_j) \;\; \text{(multilinear)}$$

So it suffices to prove an inner product over the integers.

The idea: evaluate both vectors at a very large integer α

$$f(\alpha) = \sum_{i=0}^{N-1} f_i \, \alpha^i, \qquad g^{\mathrm{rev}}(\alpha) = \sum_{i=0}^{N-1} g_{N-1-i} \, \alpha^i, \qquad \alpha \gg 2^B$$

The insight: evaluation as an inner product

We have a polynomial (multilinear or univariate), represented as a vector of coefficients or evaluations:

$$\mathbf{f} = (f_0, f_1, \ldots, f_{N-1})$$

Evaluating it at a point z is an inner product with a structured vector $\mathbf{g}$:

$$f(z) = \langle \mathbf{f}, \mathbf{g} \rangle$$

$$\mathbf{g} = (1, z, z^2, \ldots, z^{N-1}) \ \text{(univariate)}, \qquad \mathbf{g} = \bigotimes_{j=1}^{\log N} (1, z_j) \ \text{(multilinear)}$$

So it suffices to prove an inner product over the integers.

The idea: evaluate both vectors at a very large integer α

$$f(\alpha) = \sum_{i=0}^{N-1} f_i \, \alpha^i, \qquad g^{\mathrm{rev}}(\alpha) = \sum_{i=0}^{N-1} g_{N-1-i} \, \alpha^i, \qquad \alpha \gg 2^B$$

$$f(\alpha) \cdot g^{\mathrm{rev}}(\alpha) = \underbrace{L}_{\sum_{i=0}^{N-2} c_i \, \alpha^i} + \underbrace{M}_{\langle \mathbf{f}, \mathbf{g} \rangle} \cdot \alpha^{N-1} + \underbrace{R}_{\sum_{i=N}^{2N-2} c_i \, \alpha^i}$$

Commitment and evaluation

Inspired by the last slide, we commit to a polynomial f in the following form:

Evaluate at α
$f(\alpha) \in \mathbb{Z}$

→

Binary decomposition
$\mathbf{a} = \mathrm{Bits}\big(f(\alpha)\big)$

→

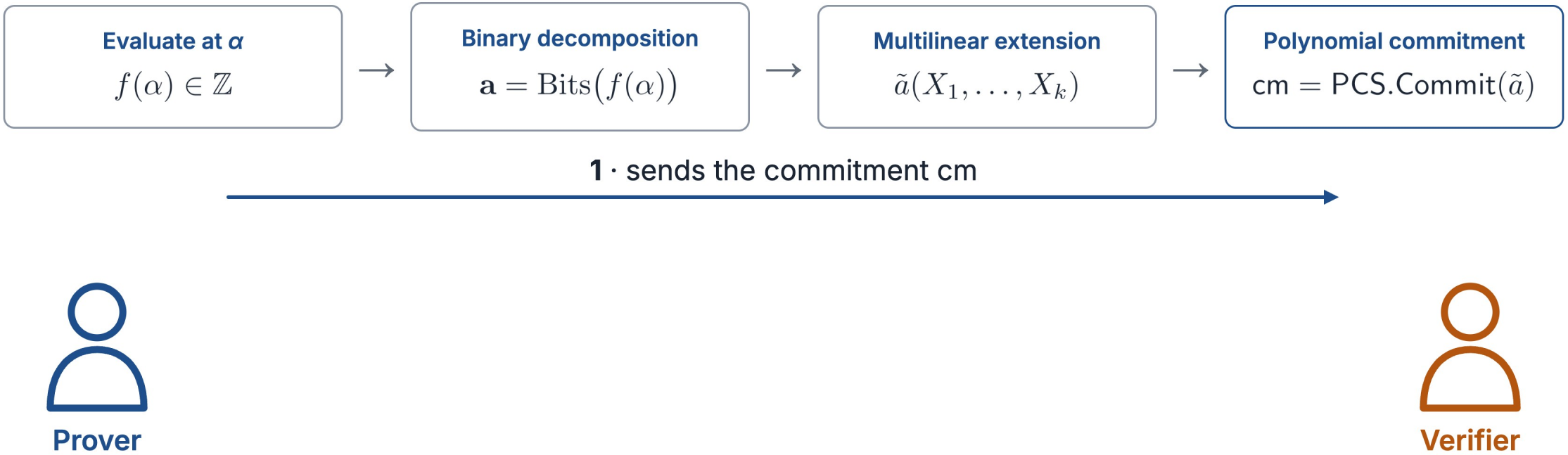
Multilinear extension
$\tilde{a}(X_1, \ldots, X_k)$

→

Polynomial commitment
$\mathsf{cm} = \mathsf{PCS.Commit}(\tilde{a})$

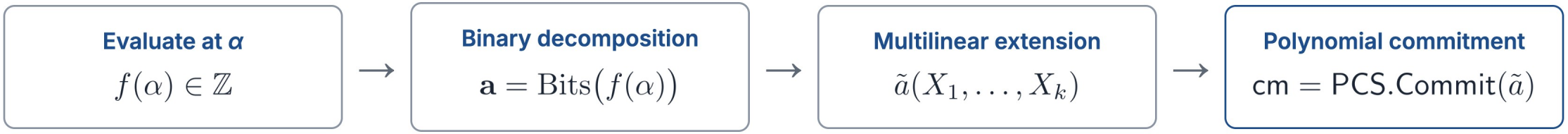
Commitment and evaluation

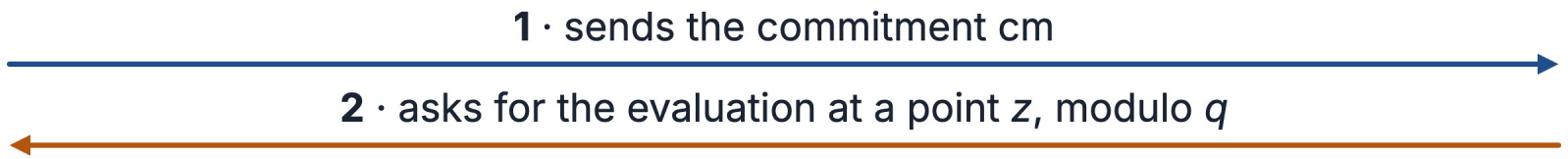
Inspired by the last slide, we commit to a polynomial f in the following form:

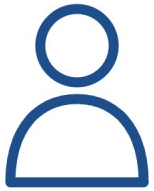


Commitment and evaluation

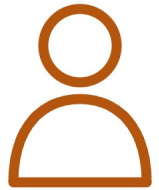
Inspired by the last slide, we commit to a polynomial f in the following form:







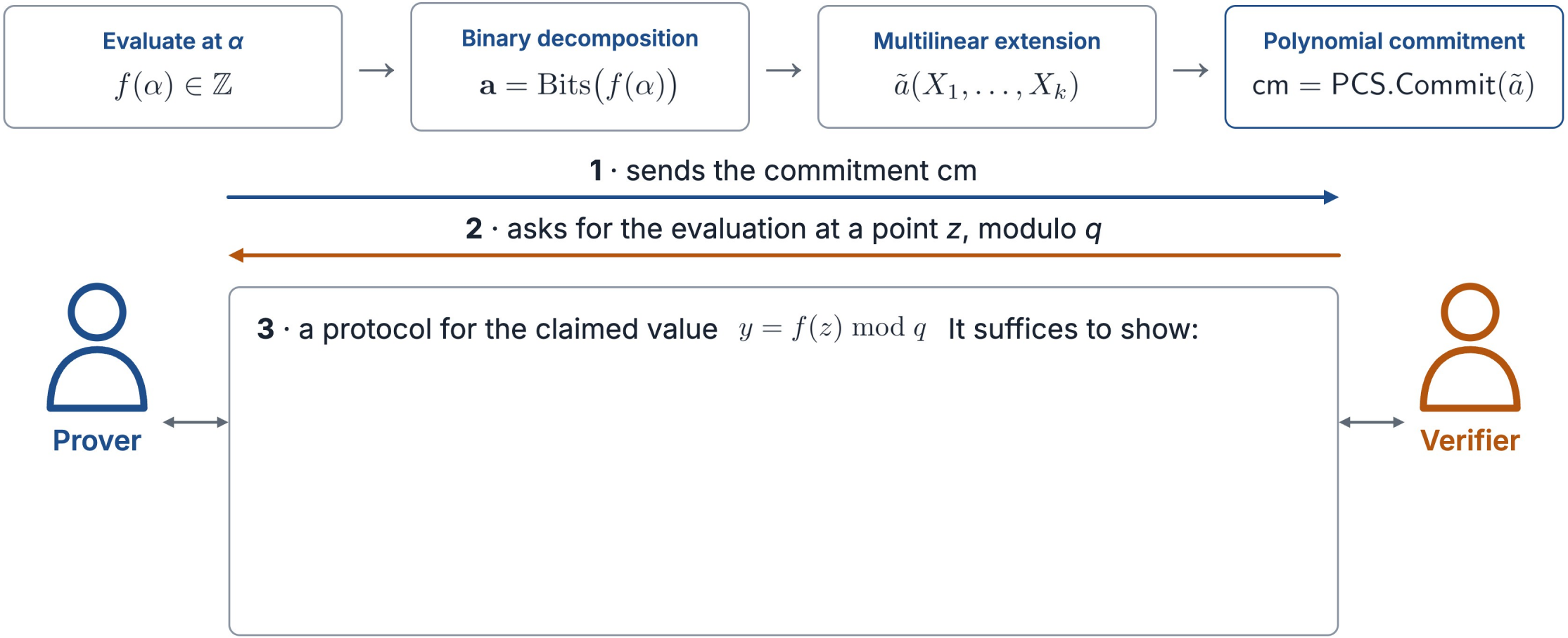
Prover



Verifier

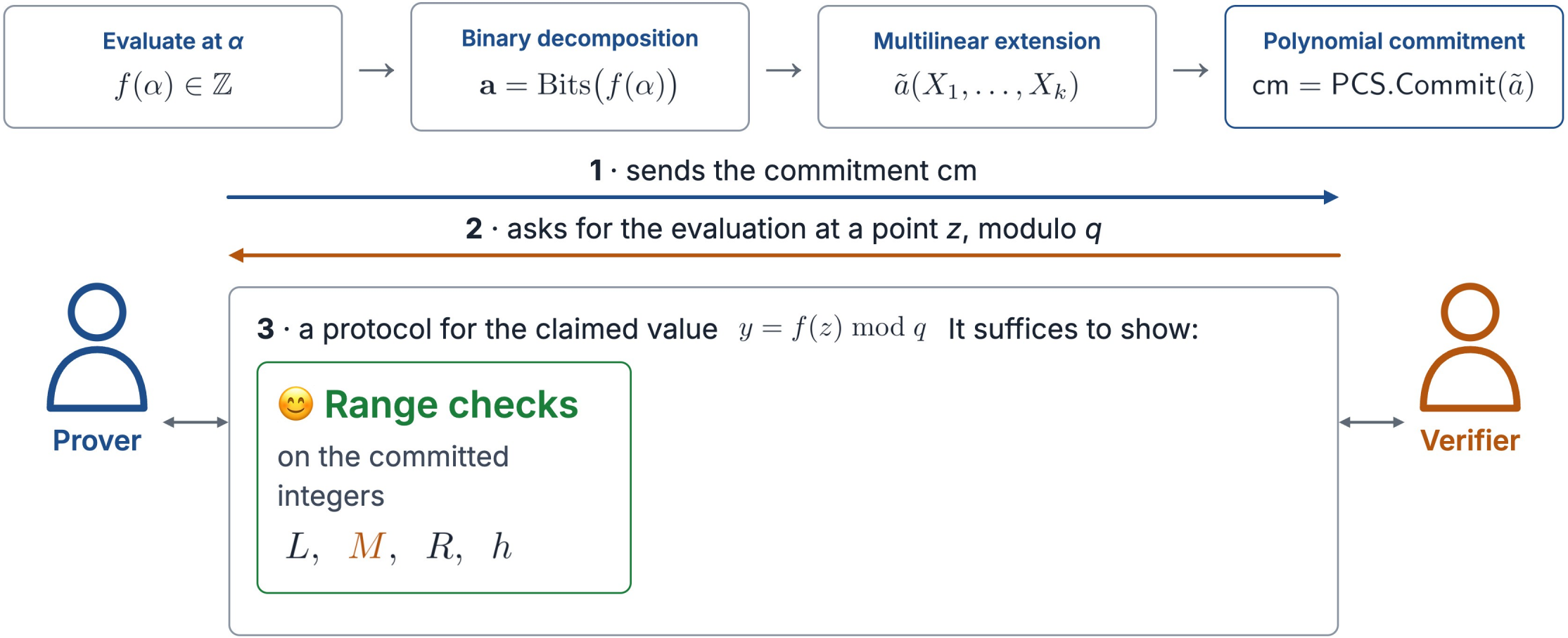
Commitment and evaluation

Inspired by the last slide, we commit to a polynomial f in the following form:



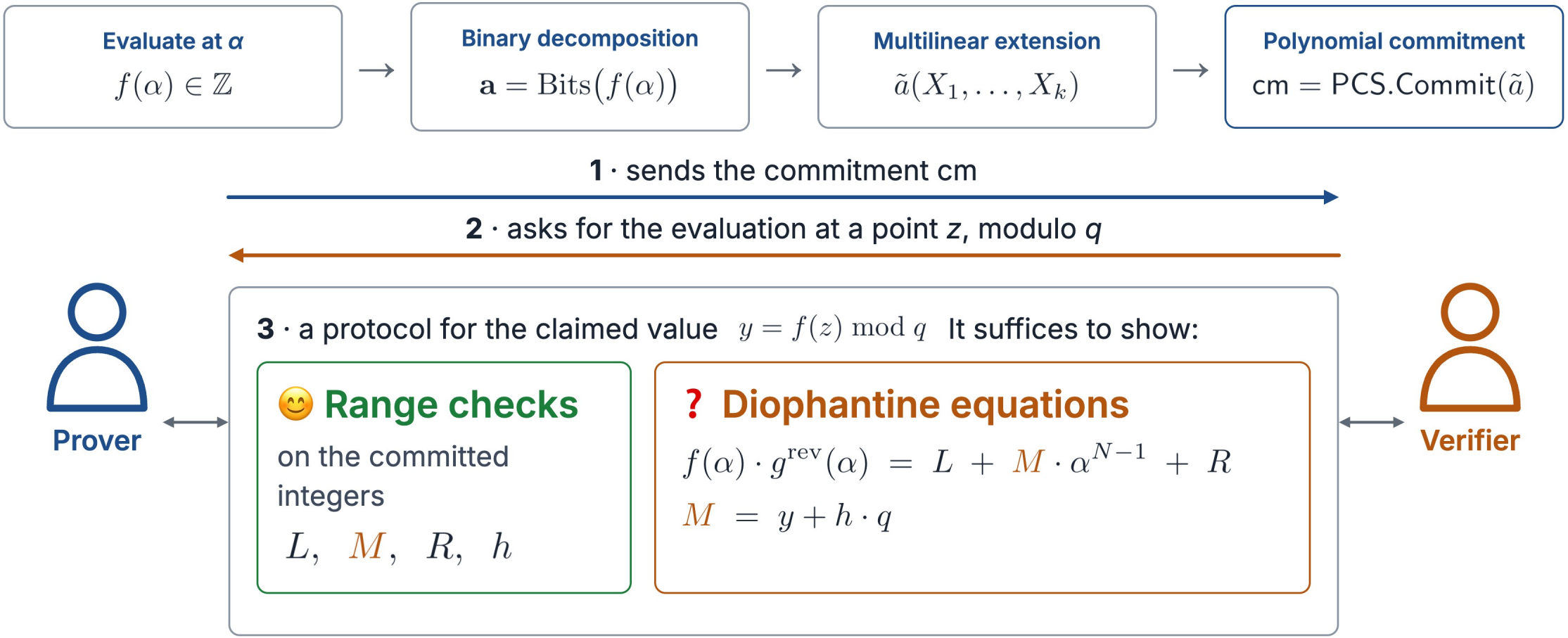
Commitment and evaluation

Inspired by the last slide, we commit to a polynomial f in the following form:



Commitment and evaluation

Inspired by the last slide, we commit to a polynomial f in the following form:



Side contributions

MaSDAK: a protocol for *B*-bit Diophantine equations

Magnitude-**S**uccinct **D**iophantine **A**rgument of **K**nowledge

$$G(a_1, \ldots, a_t) = 0 \quad \text{over } \mathbb{Z}$$

Big-O omitted. *t*: number of integers. *s*: size of the equation. *B*: bit-length bound. GUO: groups of unknown order. RO: random oracle.

Side contributions

MaSDAK: a protocol for *B*-bit Diophantine equations

Magnitude-**S**uccinct **D**iophantine **A**rgument of **K**nowledge

$$G(a_1, \ldots, a_t) = 0 \quad \text{over } \mathbb{Z}$$

Scheme	PQ	ZK	Transparent	Unbounded integers	Assumption	Prover	Verifier	Proof
TV20	✗	✓	✓	✓	GUO	sB log B	B + log s + λ	B + log s + λ
PoKEMath, BMSS25	✓	✗	✓	✓	GUO	tB log B	λ(t + s log λ)	tλ
MaSDAK + TensorSwitch, BFRW25 (this work)	✓	✗	✓	✗	RO	tB log B	tλ + log tB + λ log λ	tλ + log tB

Big-O omitted. *t*: number of integers. *s*: size of the equation. *B*: bit-length bound. GUO: groups of unknown order. RO: random oracle.

Side contributions

MaSDAK: a protocol for *B*-bit Diophantine equations

Magnitude-**S**uccinct **D**iophantine **A**rgument of **K**nowledge

$$G(a_1, \ldots, a_t) = 0 \quad \text{over } \mathbb{Z}$$

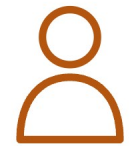
Scheme	PQ	ZK	Transparent	Unbounded integers	Assumption	Prover	Verifier	Proof
TV20	✗	✓	✓	✓	GUO	sB log B	B + log s + λ	B + log s + λ
PoKEMath, BMSS25	✓	✗	✓	✓	GUO	tB log B	λ(t + s log λ)	tλ
MaSDAK + TensorSwitch, BFRW25 (this work)	✓	✗	✓	✗	RO	tB log B	tλ + log tB + λ log λ	tλ + log tB

1 · sends a random λ-bit prime ℓ





Prover



Verifier

Big-O omitted. *t*: number of integers. *s*: size of the equation. *B*: bit-length bound. GUO: groups of unknown order. RO: random oracle.

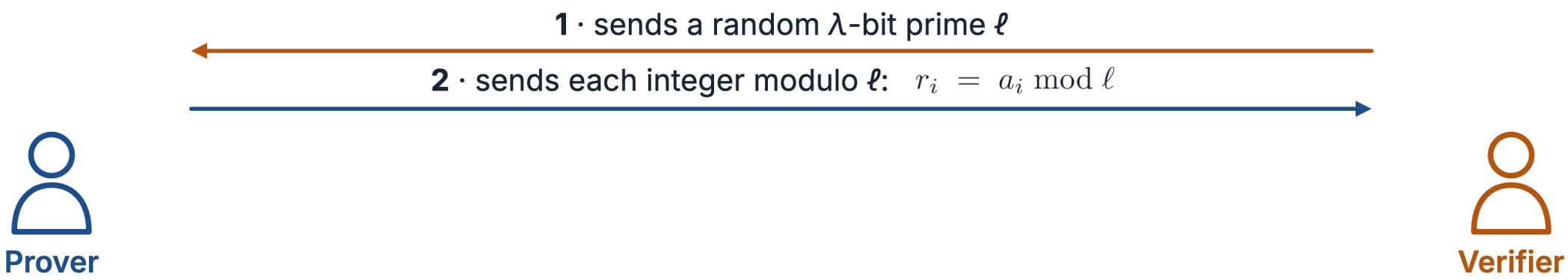
Side contributions

MaSDAK: a protocol for *B*-bit Diophantine equations

Magnitude-**S**uccinct **D**iophantine **A**rgument of **K**nowledge

$$G(a_1, \ldots, a_t) = 0 \quad \text{over } \mathbb{Z}$$

Scheme	PQ	ZK	Transparent	Unbounded integers	Assumption	Prover	Verifier	Proof
TV20	✗	✓	✓	✓	GUO	sB log B	B + log s + λ	B + log s + λ
PoKEMath, BMSS25	✓	✗	✓	✓	GUO	tB log B	λ(t + s log λ)	tλ
MaSDAK + TensorSwitch, BFRW25 (this work)	✓	✗	✓	✗	RO	tB log B	tλ + log tB + λ log λ	tλ + log tB



Big-O omitted. *t*: number of integers. *s*: size of the equation. *B*: bit-length bound. GUO: groups of unknown order. RO: random oracle.

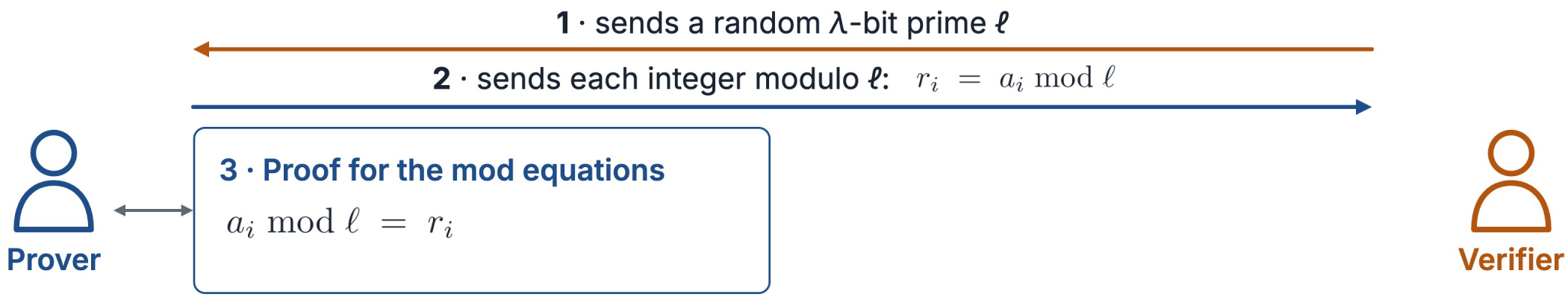
Side contributions

MaSDAK: a protocol for *B*-bit Diophantine equations

Magnitude-**S**uccinct **D**iophantine **A**rgument of **K**nowledge

$$G(a_1, \ldots, a_t) = 0 \quad \text{over } \mathbb{Z}$$

Scheme	PQ	ZK	Transparent	Unbounded integers	Assumption	Prover	Verifier	Proof
TV20	✗	✓	✓	✓	GUO	sB log B	B + log s + λ	B + log s + λ
PoKEMath, BMSS25	✓	✗	✓	✓	GUO	tB log B	λ(t + s log λ)	tλ
MaSDAK + TensorSwitch, BFRW25 (this work)	✓	✗	✓	✗	RO	tB log B	tλ + log tB + λ log λ	tλ + log tB



Big-O omitted. *t*: number of integers. *s*: size of the equation. *B*: bit-length bound. GUO: groups of unknown order. RO: random oracle.

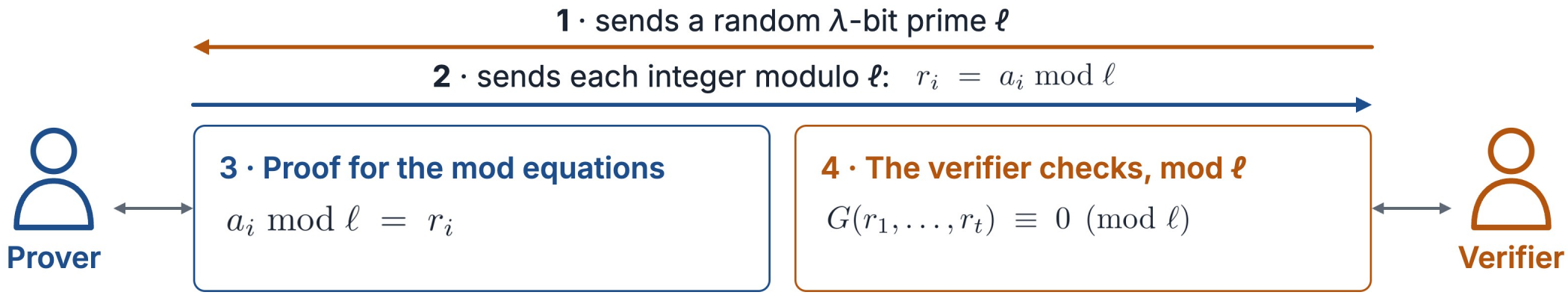
Side contributions

MaSDAK: a protocol for *B*-bit Diophantine equations

Magnitude-**S**uccinct **D**iophantine **A**rgument of **K**nowledge

$$G(a_1, \ldots, a_t) = 0 \quad \text{over } \mathbb{Z}$$

Scheme	PQ	ZK	Transparent	Unbounded integers	Assumption	Prover	Verifier	Proof
TV20	✗	✓	✓	✓	GUO	sB log B	B + log s + λ	B + log s + λ
PoKEMath, BMSS25	✓	✗	✓	✓	GUO	tB log B	λ(t + s log λ)	tλ
MaSDAK + TensorSwitch, BFRW25 (this work)	✓	✗	✓	✗	RO	tB log B	tλ + log tB + λ log λ	tλ + log tB



Big-O omitted. *t*: number of integers. *s*: size of the equation. *B*: bit-length bound. GUO: groups of unknown order. RO: random oracle.

The modularity check

To check $a_i \bmod \ell = r_i$

Add: integer addition. Mul: integer multiplication. Big-O omitted. *t*: number of instances. *B*: bit-length bound.

The modularity check

To check $a_i \bmod \ell = r_i$ write it with a quotient:

$$a_i = \underbrace{h_i \cdot \ell}_{\text{multiplication}} \underbrace{+ r_i}_{\text{addition}}$$

Add: integer addition. Mul: integer multiplication. Big-O omitted. *t*: number of instances. *B*: bit-length bound.

The modularity check

To check $a_i \bmod \ell = r_i$ write it with a quotient: $a_i = \underbrace{h_i \cdot \ell}_{\text{multiplication}} \underbrace{+ r_i}_{\text{addition}}$

If we can prove integer addition and multiplication over committed integers, we are done. We design protocols for both that operate on the bit representations.

Add: integer addition. Mul: integer multiplication. Big-O omitted. *t*: number of instances. *B*: bit-length bound.

The modularity check

To check $a_i \bmod \ell = r_i$ write it with a quotient: $a_i = \underbrace{h_i \cdot \ell}_{\text{multiplication}} \underbrace{+ r_i}_{\text{addition}}$

If we can prove integer addition and multiplication over committed integers, we are done. We design protocols for both that operate on the bit representations.

Scheme	PQ	ZK	Assumption	Add: prover	Add: verifier	Add: proof	Mul: prover	Mul: verifier	Mul: proof
LLNW18	✓	✓	SIVP	tB	tB	tB	tB^($\log_2 3$)	tB^($\log_2 3$)	tB^($\log_2 3$)
LNS20	✓	✓	R-SIS, R-LWE	tB	tB	tB	tB log B	tB	tB
KSSL21	✓	✓	M-SIS, M-LWE	tB	tB	tB	tB log B	tB log B	tB log B
This work + TensorSwitch, BFRW25	✓	✗	RO	tB	log tB	log tB	tB log B	log tB	log tB

Add: integer addition. Mul: integer multiplication. Big-O omitted. *t*: number of instances. *B*: bit-length bound.

The hierarchy of our protocols

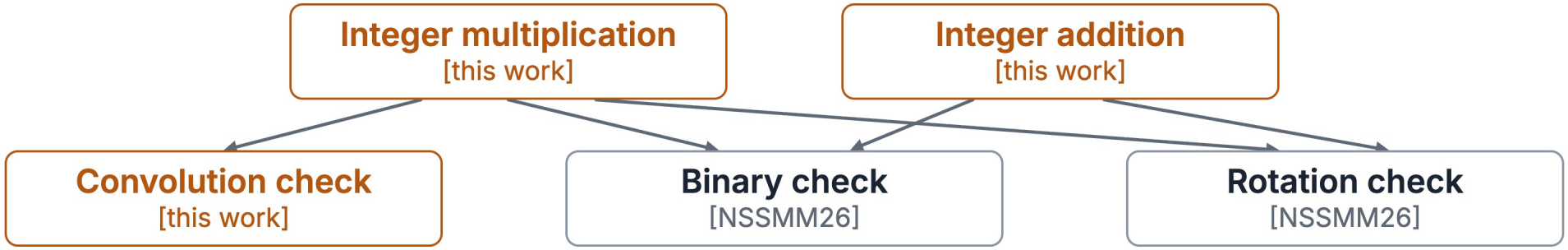
Integer multiplication and addition reduce, protocol by protocol, to sumcheck. Sumcheck leaves only evaluation claims.

Integer multiplication
[this work]

Integer addition
[this work]

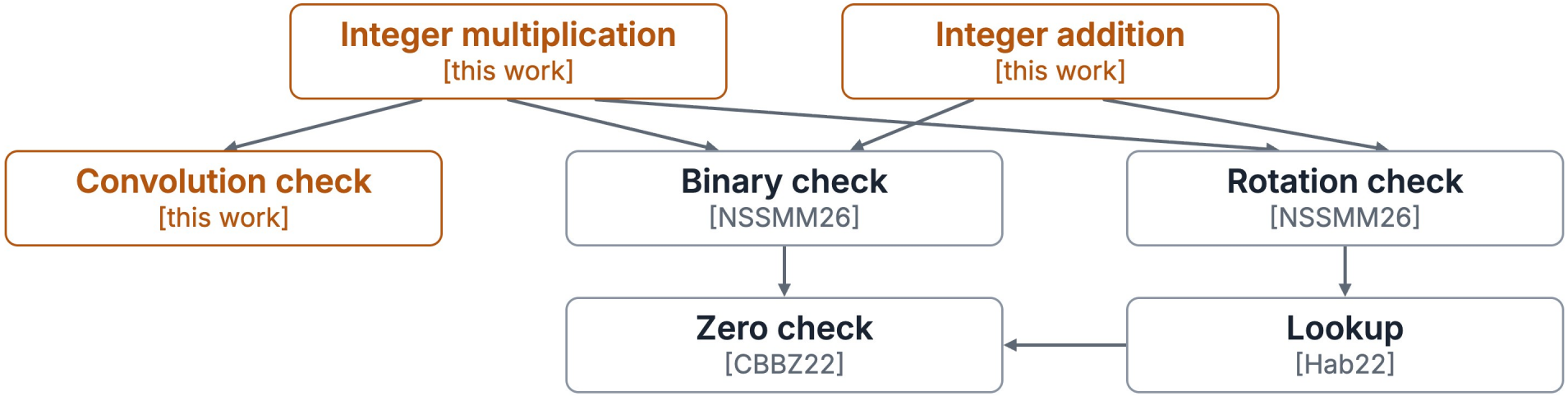
The hierarchy of our protocols

Integer multiplication and addition reduce, protocol by protocol, to sumcheck. Sumcheck leaves only evaluation claims.



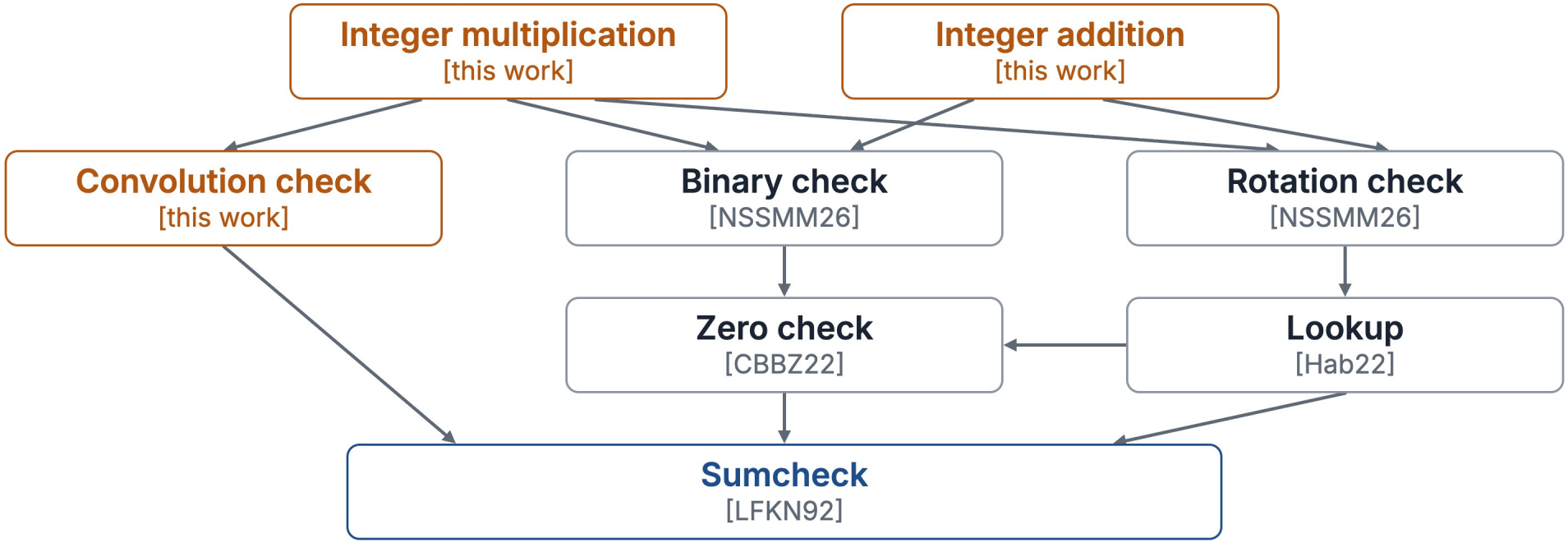
The hierarchy of our protocols

Integer multiplication and addition reduce, protocol by protocol, to sumcheck. Sumcheck leaves only evaluation claims.



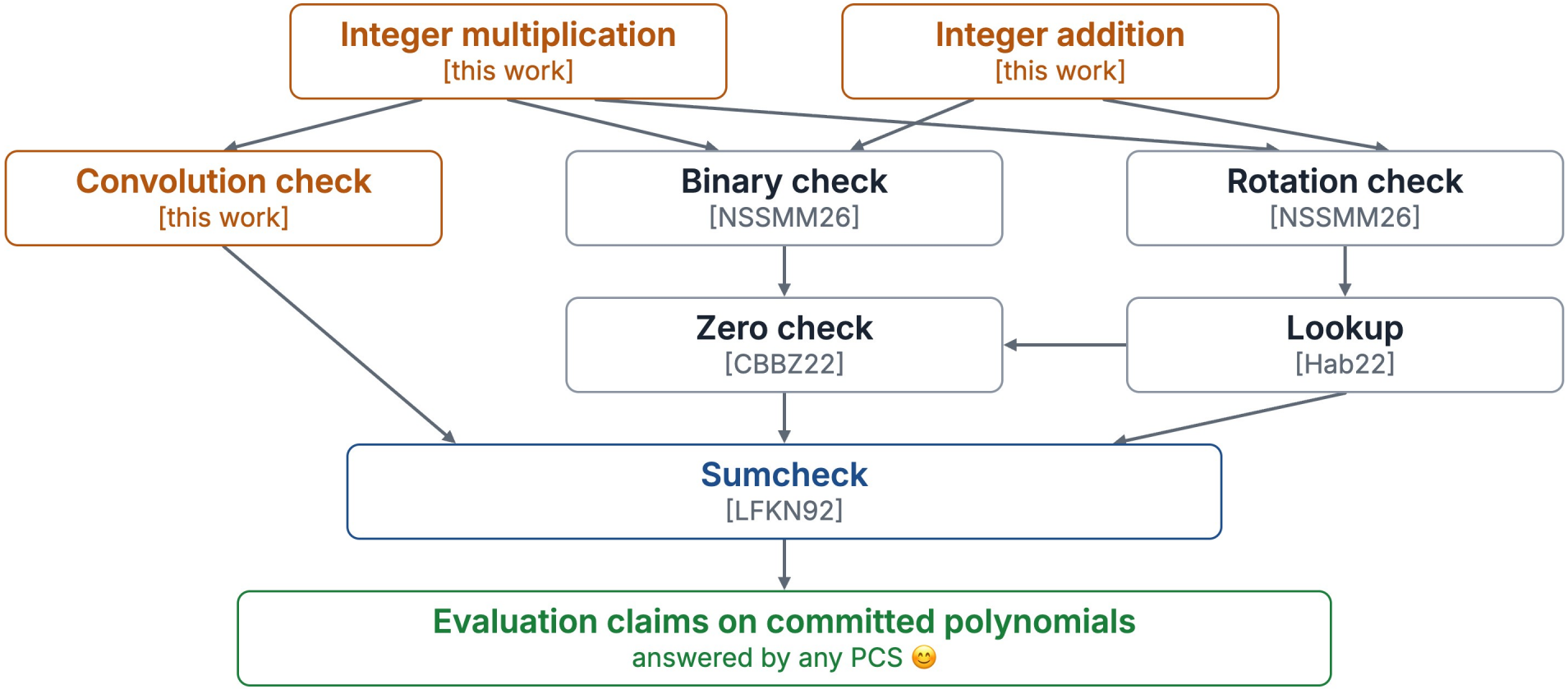
The hierarchy of our protocols

Integer multiplication and addition reduce, protocol by protocol, to sumcheck. Sumcheck leaves only evaluation claims.



The hierarchy of our protocols

Integer multiplication and addition reduce, protocol by protocol, to sumcheck. Sumcheck leaves only evaluation claims.



From mod α to mod q: commitment switching

? The problem: all our claims so far are modulo α

We have $f(z) \equiv y + h \cdot q \pmod{\alpha}$ and we need $q \mid \alpha \implies f(z) \equiv y \pmod{q}$

But α is fixed at commitment time, before the verifier picks q.

From mod α to mod q: commitment switching

? The problem: all our claims so far are modulo α

We have $f(z) \equiv y + h \cdot q \pmod{\alpha}$ and we need $q \mid \alpha \implies f(z) \equiv y \pmod{q}$

But α is fixed at commitment time, before the verifier picks q.

1 · commits with a base α that is only large: $\mathsf{cm} = \mathsf{Commit}(f(\alpha))$



From mod α to mod q: commitment switching

? The problem: all our claims so far are modulo α

We have $f(z) \equiv y + h \cdot q \pmod{\alpha}$ and we need $q \mid \alpha \implies f(z) \equiv y \pmod{q}$

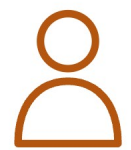
But α is fixed at commitment time, before the verifier picks q.

1 · commits with a base α that is only large: $\mathsf{cm} = \mathsf{Commit}(f(\alpha))$

2 · sends the random prime q and a point z



Prover



Verifier

From mod α to mod q: commitment switching

? The problem: all our claims so far are modulo α

We have $f(z) \equiv y + h \cdot q \pmod{\alpha}$ and we need $q \mid \alpha \implies f(z) \equiv y \pmod{q}$

But α is fixed at commitment time, before the verifier picks q.

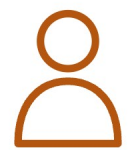
1 · commits with a base α that is only large: $\mathsf{cm} = \mathsf{Commit}(f(\alpha))$

2 · sends the random prime q and a point z

3 · commits again, and sends y: $\beta = q \cdot \alpha, \quad \mathsf{cm}' = \mathsf{Commit}(f(\beta))$



Prover



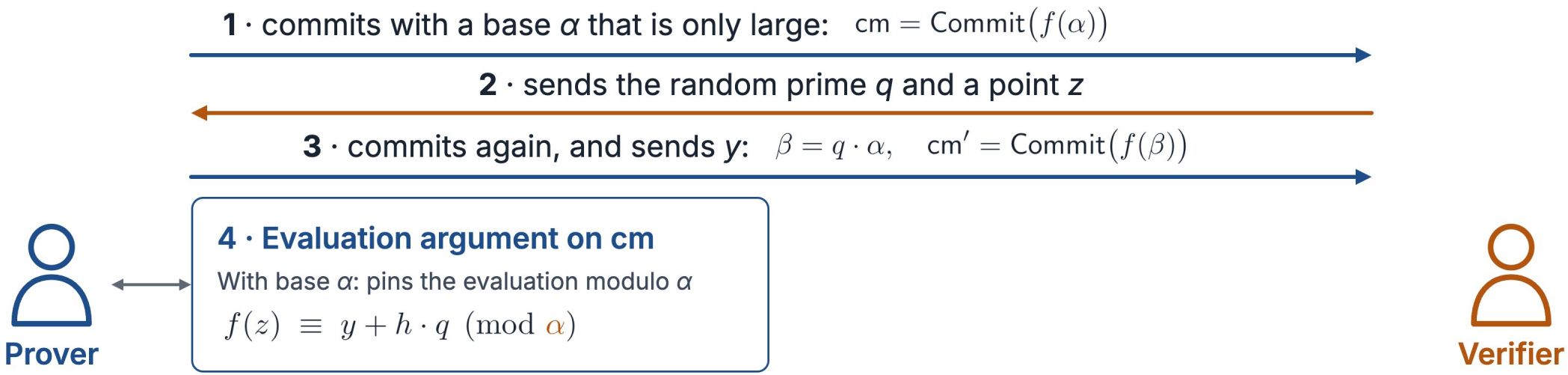
Verifier

From mod α to mod q: commitment switching

? The problem: all our claims so far are modulo α

We have $f(z) \equiv y + h \cdot q \pmod{\alpha}$ and we need $q \mid \alpha \implies f(z) \equiv y \pmod{q}$

But α is fixed at commitment time, before the verifier picks q.

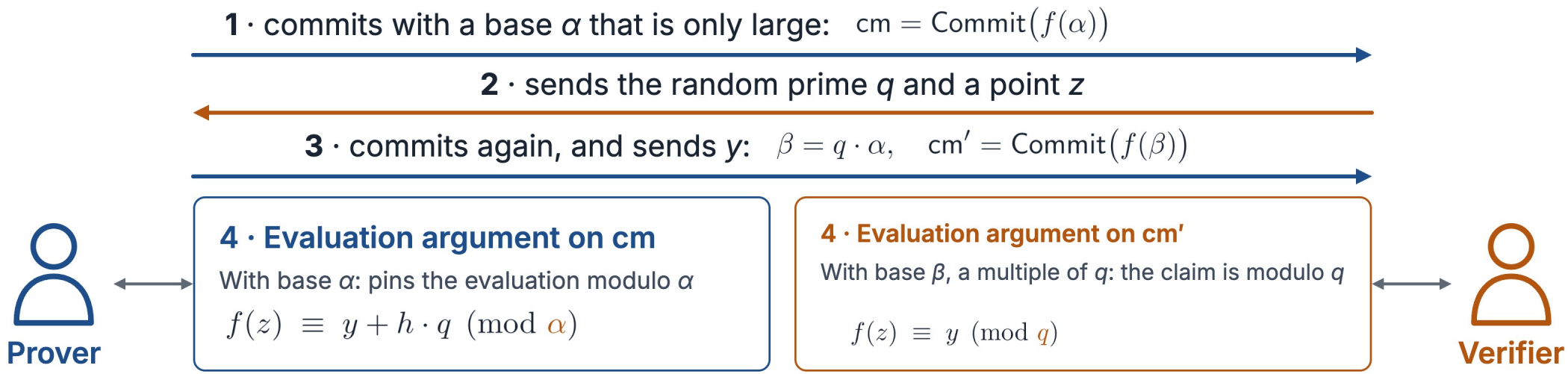


From mod α to mod q: commitment switching

? The problem: all our claims so far are modulo α

We have $f(z) \equiv y + h \cdot q \pmod{\alpha}$ and we need $q \mid \alpha \implies f(z) \equiv y \pmod{q}$

But α is fixed at commitment time, before the verifier picks q.

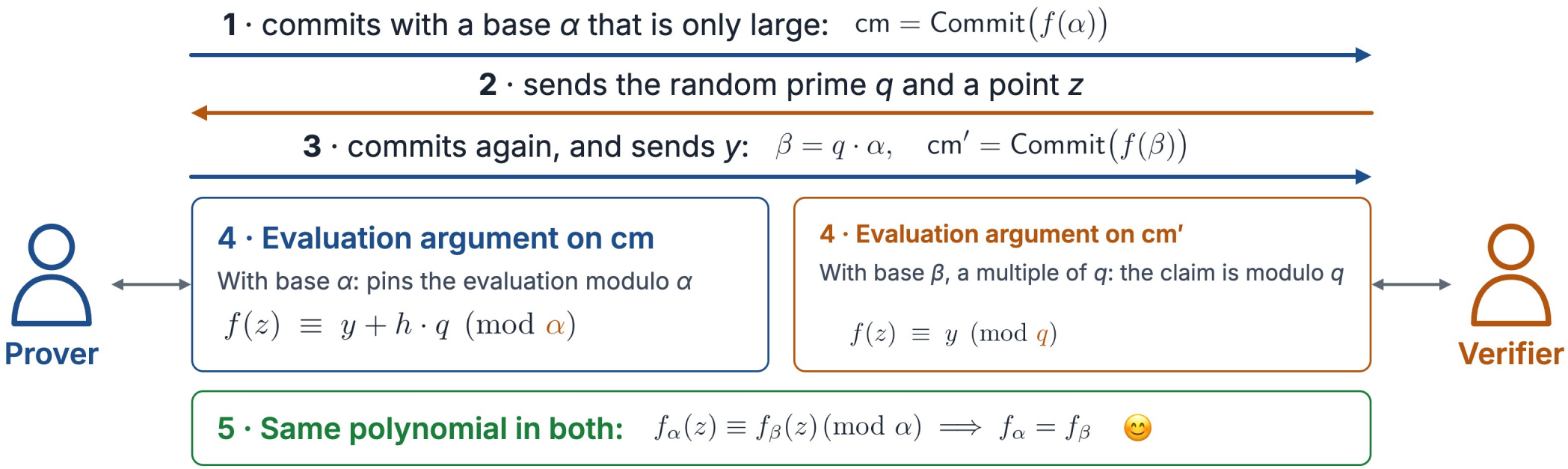


From mod α to mod q: commitment switching

? The problem: all our claims so far are modulo α

We have $f(z) \equiv y + h \cdot q \pmod{\alpha}$ and we need $q \mid \alpha \implies f(z) \equiv y \pmod{q}$

But α is fixed at commitment time, before the verifier picks q.



Thank you!

Relaxed Modular PCS from Arbitrary PCS and Applications to SNARKs for Integers

Alireza Shirzad · Sriram Sridhar · Dimitrios Papadopoulos · Charalampos Papamanthou

eprint.iacr.org/2026/347

Scan for the paper

These slides were created by AI under human supervision.